

Test Design & Test Management Fundamentals

Good test cases are not guessed, they are engineered. This module turns the ShopFast requirements into a small, traceable, fully covered set of tests.

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table
- State Transition

DURATION	TECHNIQUES	TEST CASES	HANDS-ON
75 min	4	15-20	60 min

RUNNING PROJECT

ShopFast

Checkout module, the same project from Module 1 through the capstone.

SCHEDULE SLOT

15:45 - 17:00

Day 1 · final session before the retrospective.

MODULE DELIVERABLE

Test Plan + 15-20 test cases

mapped to RTM v1.

KRISA ALIGNMENT

D12

Master Test Plan, Acceptance Testing phase deliverable, [Ch.6](#).

Jadual slot: Modul 4: Reka Bentuk Ujian dan Pengurusan Ujian

Run sheet / session timing

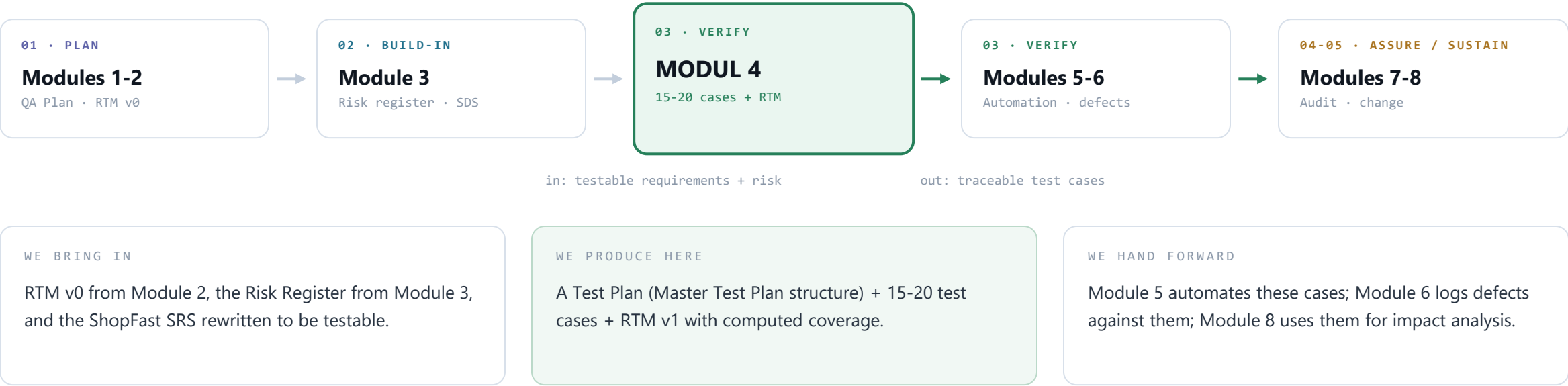
Hari 1, 15:45 hingga 17:00 · 75 minit · kira-kira 80% amali

10 min	10 min	12 min	15 min	8 min	15 min	5 min
Konsep	Klinik teknik	Makmal A	Makmal B	Makmal C	Bengkel AI W04	Rumusan

Masa	Segmen	Tempoh
15:45 hingga 15:55	Konsep	10 minit
15:55 hingga 16:05	Klinik teknik	10 minit
16:05 hingga 16:17	Makmal A	12 minit
16:17 hingga 16:32	Makmal B	15 minit
16:32 hingga 16:40	Makmal C	8 minit
16:40 hingga 16:55	Bengkel AI W04	15 minit
16:55 hingga 17:00	Rumusan	5 minit

Module 4 is where quality turns from opinion into evidence

Modules 2 and 3 produced testable requirements and prioritised risks. Module 4 converts them into concrete test cases, and everything after this refers back to them.



Four abilities, not four topics

Each outcome is checked against a real artifact you produce in the lab.

01 Pick the right test level

Place each requirement at the cheapest level that can test it, unit, integration, system, UAT or PAT, and know who owns it.

EVIDENCE

The Level column in your test case sheet.

02 Design cases by technique, not instinct

Use EP, BVA, decision tables and state transition for maximum coverage from minimum cases.

EVIDENCE

Every case names the technique that produced it.

03 Write cases someone else can run

Preconditions, test data, steps, and one observable expected result.

EVIDENCE

Peer check: another team runs 2 of your cases.

04 Prove coverage with the RTM

Link requirement to case to result to defect, compute coverage, and spot gaps early.

EVIDENCE

RTM v1 with a coverage cell and a gap list.

20% concept · 80% hands on the keyboard



TEAM ROLES (4 PEOPLE)

- Test designer, owns techniques and the case list.
- Data writer, owns test data and preconditions.
- RTM keeper, owns traceability and coverage.
- Reviewer, tries to run another team’s cases.

TOOLING (PICK ONE)

[TestRail](#) · Zephyr · Xray · [Jira](#) · Google Sheets (default).

A sheet is enough today. What is assessed is structure and traceability, not the tool.

WORKING RULE

No test case gets written without a requirement ID beside it. If you cannot name the requirement being tested, the case is not ready.

Every deliverable has a test that **proves it**

The left arm produces documents; the right arm proves them. In KRISA, acceptance testing splits into UAT (functional) and PAT (functional and non-functional).

dashed line = this document is the basis for the test cases on the right



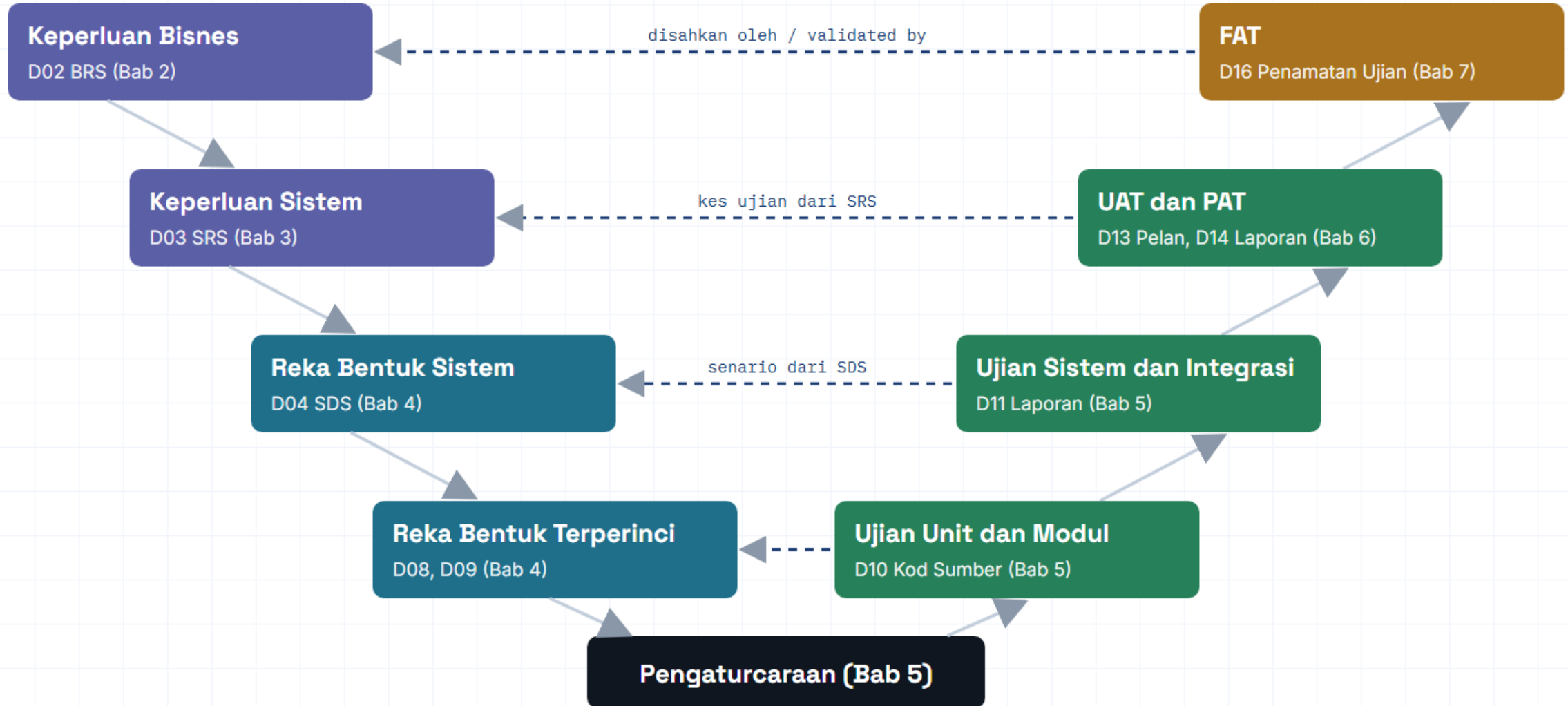
TODAY'S LAB

The cases you write in Lab B sit at the system test level, detailed enough for Module 5 automation, and the source for UAT scenarios later.

Model V: setiap dokumen ada ujian yang membuktikannya

V-model: every document has a test that proves it

Peringkat ujian KRISAv2: Bab 5 (Unit, Integrasi, Sistem) dan Bab 6 hingga 7 (UAT, PAT, FAT)



Verifikasi: membina sistem dengan betul

Sumber: KRISAv2 Beta 2026 Bab 1 (Rajah 1.11, 1.12), Bab 5 5.7, Bab 6 Jadual 6.2, Bab 7 7.6; nombor templat ikut fail D01

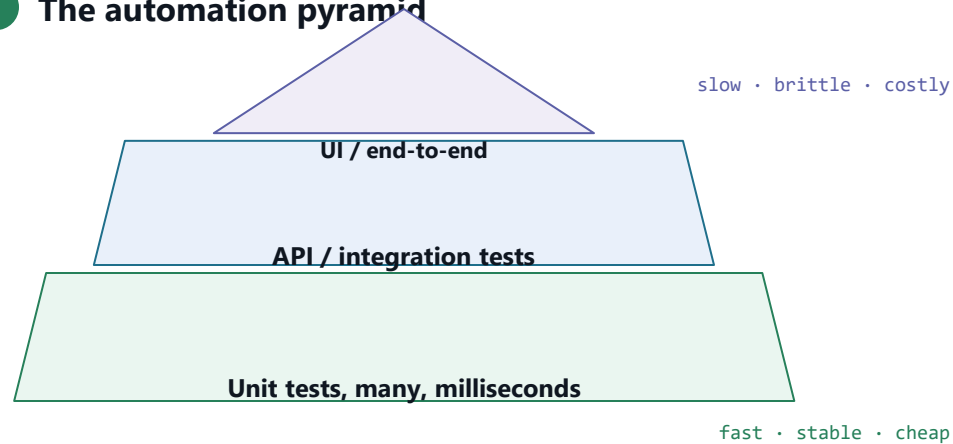
Module hub

Open: KRISAv2 Bab 6 p.5

How many tests sit at each level decides **your feedback speed**

TARGET

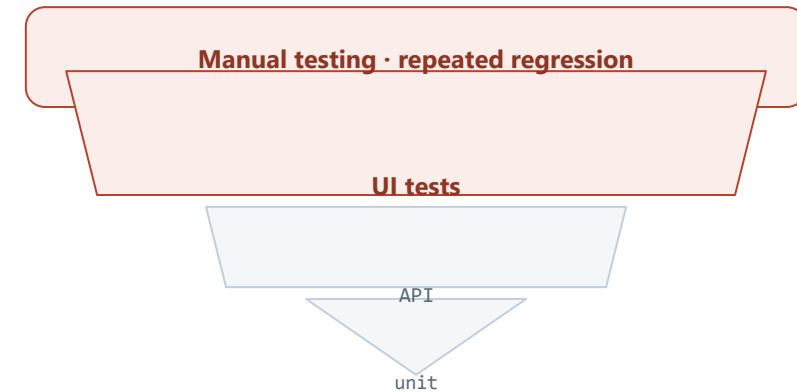
The automation pyramid



Module 5 automates only 1-2 UI paths. The rest drops to the API level.

ANTI-PATTERN

The ice-cream cone



Symptoms: regression takes days, release dates slip, testers become the bottleneck.

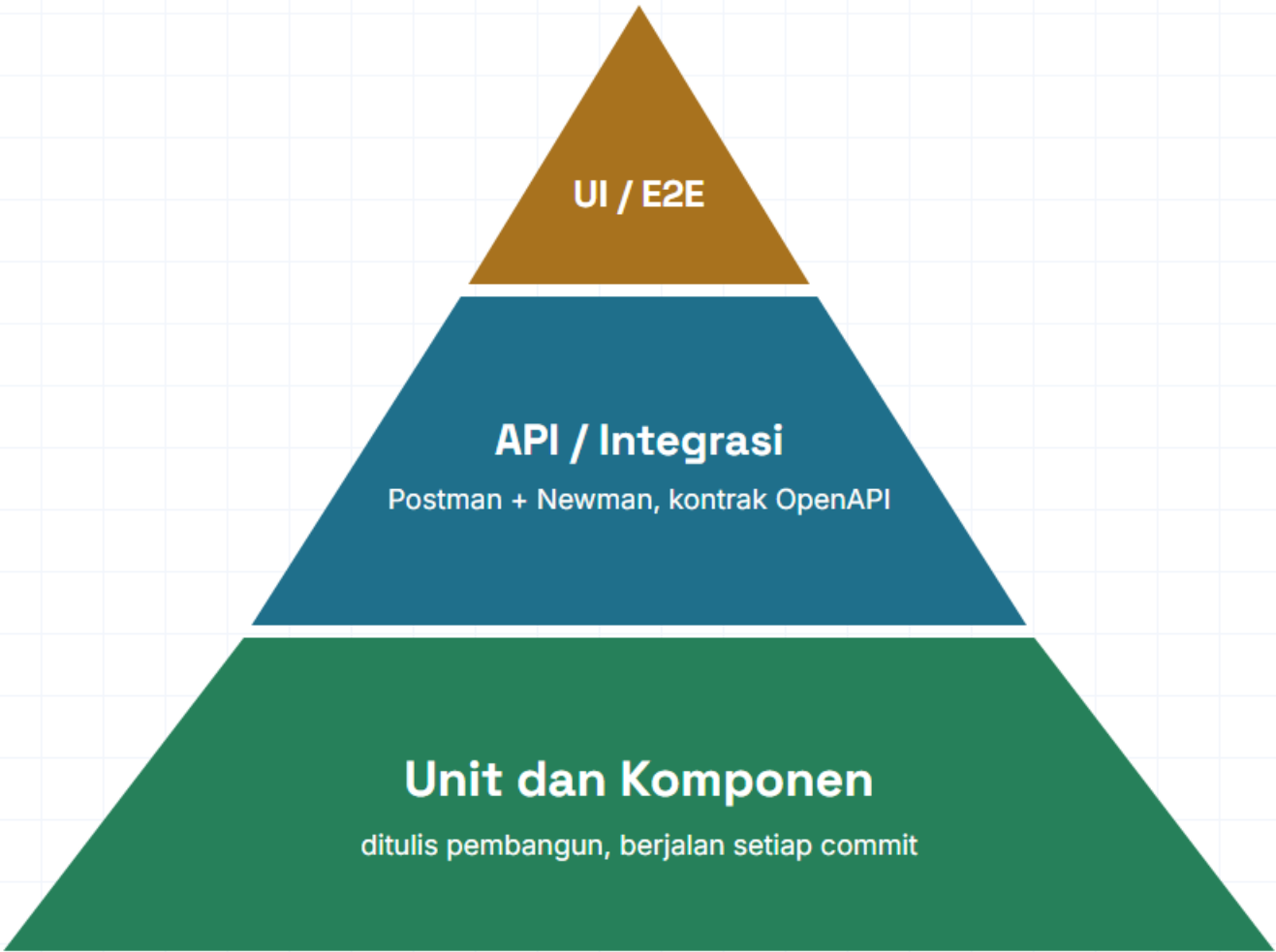
KRISA LINK

[KRISA 2.0 Ch.8](#) (DevOps alignment) calls for automation and continuous monitoring, the pyramid shape is what lets a release pipeline run without days of manual testing.

Piramid automasi ujian: di mana masa dan kos patut pergi

Test automation pyramid: where time and money should go

Banyak ujian unit yang pantas di bawah, sedikit ujian UI yang perlahan di atas. Elak bentuk kon ais krim.



Lapisan	Bil. ujian	Masa	Contoh alat
UI / E2E	5 hingga 10%	minit	Playwright, Selenium
API	20 hingga 30%	saat	Postman, Newman, k6
Unit	60 hingga 70%	milisaat	Jest, pytest, JUnit

KRISAv2 Bab 7.10: ujian SIT, UAT dan prestasi dijalankan di persekitaran **staging**. Bab 8: automasi dalam saluran CI/CD sebelum Laporan Release.

Quality gates: when testing starts, and when it is finished

Without both, testing starts too early and ends when time runs out, not when risk is closed.

Build ready · SUT deployed

staging environment

ENTRY GATE

Every entry checklist item = YES
if not, testing does not start

Test cycle

execute · log · retest

EXIT GATE

Release decision: go / no-go
backed by the Test Closure Report

ENTRY

Examples (KRISA, UAT)

- The UAT plan has been approved by the project team.
- The staging environment is ready and the SUT is deployed in it.
- Test scenarios and scripts are prepared and approved.
- Test data has been loaded into the SUT database.
- System test results accepted; testers have been briefed.

EXIT

Examples (KRISA, UAT)

- All severity 1 and 2 defects are fixed, retested and closed.
- The UAT closure report is approved by the project owner.
- Remaining open defects accepted in writing with a workaround.
- High-risk requirement coverage reaches 100% in the RTM.

PAT adds non-functional aspects, performance, security, usability, to the same criteria.

D12 Master Test Plan, one document, five decisions

You are not writing the full document today. You fill the five sections that carry all the real decisions.

Section	Decision it records	Source in your ShopFast work
1 • Project introduction	Purpose, scope, stakeholders, glossary	QA Plan, Module 2
2 • Test context	Test items, test levels, entry & exit criteria	Previous slide + the SRS
3 • Test strategy	Test types, design techniques, environment, data	Technique clinic + Lab A
4 • Test risk register	Product and project risks + mitigation	Risk Register, Module 3
5 • Test deliverables	What is handed over: cases, reports, data, logs	Lab B + Lab C

VERIFIED KRISA REFERENCES

- [D12](#) Master Test Plan is listed as an Acceptance Testing phase deliverable ([Ch.6](#)).
- Processes: F5.1 Master Test Plan · F5.3 UAT/PAT plans · F5.4 UAT · F5.5 PAT · F5.6 Acceptance Test Report.
- References [D03](#) SRS · [D04](#) SDS · [D08](#) Integration Spec · [D11](#) System Test Report.

VERSION CAUTION

KRISA 2.0 is still Beta 2026. Deliverable and template numbering (D-, F-, T-) is still being refined. Confirm current numbering with your agency before submitting a real document.

TASK

Each team fills Sections 2, 3 and 5 before 16:17, one paragraph each is enough. Sections 1 and 4 are copied from Modules 2 and 3.

You cannot test everything, so pick the tests that **remove the most uncertainty**

A single 1-10 quantity field already has billions of inputs. Test design techniques bring that down to a handful of cases, without losing a single class of bug.

BLACK BOX

Specification-based. You do not see the code, you test the behaviour promised to the user.

- Equivalence Partitioning, classes of input that should behave the same
- Boundary Value Analysis, the edges of each class
- Decision Table, combinations of business conditions
- State Transition, valid / invalid sequences and transitions

These four = today's lab.

WHITE BOX

Structure-based. Measured by coverage tools in the pipeline, usually owned by developers.

- Statement coverage, every line executed at least once.
- Branch coverage, every if tested both true and false.
- 100% statement coverage is reachable without ever testing an else branch, so branch coverage is stronger.

Common CI threshold: fail the build if branch coverage drops.

EXPERIENCE-BASED

Complements structured techniques, it finds what the specification never mentioned.

- Exploratory testing, timeboxed sessions with a clear charter.
- Error guessing, a list of usual failures: empty fields, double clicks, expired sessions.

Keep the last 10 minutes of Lab B for one exploratory session.

RULE

Structured techniques give you coverage you can defend in an audit. Experience-based testing gives you the bugs nobody predicted. You need both.

Group the inputs that **should behave identically**, then test one from each group

REQ-CHK-01 · Item quantity must be an integer between 1 and 10.



one representative per partition, 4 values stand in for billions of inputs

THE RULE

If two values take the same code path and produce the same message, testing both adds no information.

COMMON TRAP

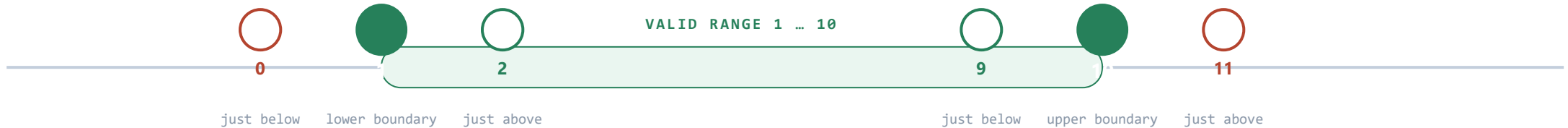
Only partitioning the valid side. Most production bugs live in the invalid partitions, especially empty input and wrong formats.

HOW TO ID IT

Name the partition (P1...P4) in the test case. When the requirement changes in Module 8, you know which cases to revisit.

Bugs cluster at the **edges**, not in the middle

EP tells you which groups exist. BVA tests the exact line where a developer typed $<$ when they meant $\leq$.



3 values per boundary = 6 cases · 2 values per boundary (0,1 and 10,11) = 4 cases, enough when time is short

THE BUG BVA CATCHES, AND NOTHING ELSE DOES

```
// ShopFast · CartService.java
if (qty > 0 && qty < 10) { addToCart(qty); }
```

A value of 5 passes. A value of 11 correctly fails. Only the value 10 exposes this bug.

BOUNDARIES ARE NOT ONLY NUMBERS

- Text length, promo codes of 5, 6, 10, 11 characters.
- Dates, a card expiring this month vs last month.
- Money, subtotals of RM199.99, RM200.00, RM200.01.
- Lists, empty cart, 1 item, maximum items.

Four checkout fields to ten test cases

Field / requirement	Valid partition	Invalid partitions	Boundary values tested	Cases
REQ-CHK-01 Item quantity	integer 1-10	≤ 0 · ≥ 11 · non-integer	0 · 1 · 10 · 11	4
REQ-CHK-02 Promo code	6-10 chars, uppercase	empty · 5 chars · 11 chars	SAVE5 · SAVE10 · SAVE123456	3
REQ-CHK-03 Free-shipping threshold	subtotal $\geq$ RM200	subtotal < RM200	199.99 · 200.00	2
REQ-CHK-04 Card expiry	current month or later	last month · wrong format	current month · last month	1

HOW THE COUNT WORKS

Boundary values absorb the partition representatives, 0 already tests P1, so you do not also need -5. That is why 4 fields yield 10 cases, not 24.

WHAT IS STILL MISSING

These ten cases test fields in isolation. Business rules that combine fields need a decision table, next slide.

INTO THE RTM

Four requirement rows, ten test case IDs. Coverage at this point: 4 of 6 checkout requirements = 67%.

When business rules overlap, tabulate every combination

Members 10%. Orders of RM200+: free shipping. Promo code 15%. Total discount never exceeds 25%. Three conditions = eight combinations.

Condition / action	R1	R2	R3	R4	R5	R6	R7	R8
C1 · Customer is a member	Y	Y	Y	Y	N	N	N	N
C2 · Subtotal ≥ RM200	Y	Y	N	N	Y	Y	N	N
C3 · Valid promo code entered	Y	N	Y	N	Y	N	Y	N
A1 · Apply 10% member discount	•	•	•	•				
A2 · Free shipping	•	•			•	•		
A3 · Apply 15% promo discount	•		•		•		•	
A4 · Cap total discount at 25%	•							
A5 · Show rejected-code error		○		○		○		○

PRIORITISE WHEN TIME IS SHORT

Test R1 first, it alone triggers the 25% cap and the triple-discount combination. Then R5, R2, R8.

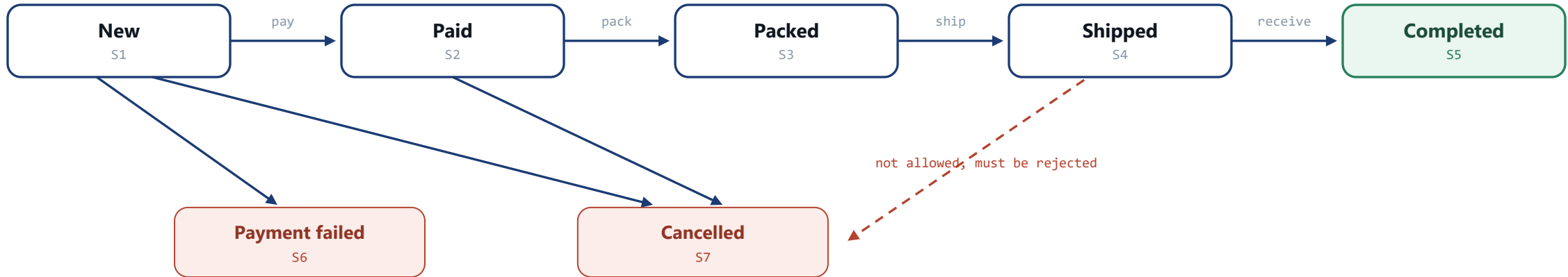
WHAT THE TABLE EXPOSES

The spec never says what happens when member + promo discount exceeds 25% and free shipping applies. That is a question for the product owner, not a guess.

• vs ○

• = the action happens. ○ = only when a code was entered and rejected. If no code is entered at all, no error is shown, a separate test case.

Test the **sequence**, not just the screen



0-switch coverage = every valid transition taken once (7 cases). Add 2 cases for transitions that must be rejected.

VALID CASES TO WRITE

S1 to S2 · S2 to S3 · S3 to S4 · S4 to S5 · S1 to S6 · S1 to S7 · S2 to S7
 Seven valid transitions = seven baseline cases.

INVALID CASES, WHERE THE REAL BUGS ARE

Cancel an order that is already Shipped (must fail) · skip New to Shipped through the API · pay a Completed order again.

Read the requirement, recognise its shape, pick the technique

If the requirement sounds like...	Use	Because	ShopFast example
between X and Y · at least · maximum	EP + BVA	A range has edges, and edges are what break.	REQ-CHK-01, 03
must be in the format · valid / invalid · from a list	EP	Values cluster into accepted or rejected classes.	REQ-CHK-02, 04
if ... and ... then · unless · except when	Decision table	Combinations of conditions produce different outcomes.	REQ-CHK-03
status moves from ... to · only before	State transition	Validity depends on what happened before.	REQ-CHK-05
within N seconds · N concurrent users · complies	Non-functional test	Measured with a tool, not manual steps, Module 5.	REQ-CHK-06
no written requirement at all	Exploratory	You are testing to discover the missing requirement.	10-minute session

CASE BUDGET

Lab B target: 8 cases from EP/BVA · 5 from the decision table (R1, R2, R5, R7, R8) · 4 from state transition · 1-3 exploratory = 18-20 cases.

A test case is an **experiment**, one question, one observable answer

RUNNABLE

Field	Content
ID	TC-CHK-004
Requirement	REQ-CHK-01 · P2 / BVA
Title	Quantity 10 (upper boundary) is accepted into the cart
Precondition	Logged in as a standard customer; cart empty; SKU-88 stock 50
Test data	SKU-88, qty = 10
Steps	1. Open the SKU-88 product page. 2. Set quantity to 10. 3. Click Add to cart.
Expected result	Cart shows SKU-88 with quantity 10; subtotal = 10 × list price; no error message.
Level · Priority	System test · High (revenue risk)

NOT RUNNABLE, AND WHY

- Check the quantity field to check for what? no question asked.
- Try different quantities to no two people run the same test.
- System works correctly to cannot objectively pass or fail.

ID CONVENTION (USE THIS TODAY)

REQ-CHK-01 to requirement

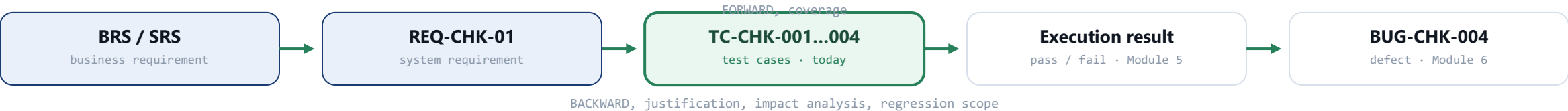
TC-CHK-004 to test case

BUG-CHK-004 to defect (Module 6)

One shared prefix across all three keeps the RTM sortable and makes Module 8 impact analysis trivial.

The RTM answers the two questions every auditor asks

Forward: does every requirement have a test? Backward: why does this test exist?



Req ID	Risk	Test cases	Technique	Status
REQ-CHK-01	High	TC-001...004	EP + BVA	Covered
REQ-CHK-02	Medium	TC-005...007	EP	Covered
REQ-CHK-03	High	TC-008...012	Decision table	Covered
REQ-CHK-04	High	TC-013	EP	Thin, no negative case
REQ-CHK-05	Medium	TC-014...017	State transition	Covered
REQ-CHK-06	High		non-functional	GAP

REQUIREMENT COVERAGE

= requirements with ≥ 1 case ÷ total requirements

= 5 ÷ 6 = 83%

83% is not a failure, it is a recorded decision. REQ-CHK-06 needs a performance tool, so it moves to Module 5.

TWO GAPS TO HUNT

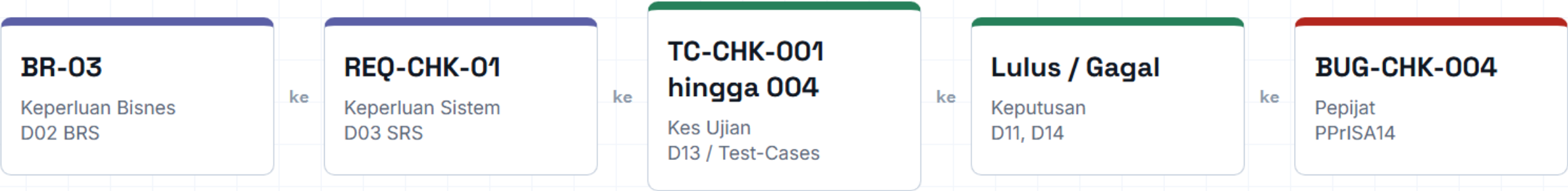
A requirement with no case = untested risk.

A case with no requirement = an orphan test; either the requirement is missing from the SRS, or the test is not needed.

Matriks kebolehhesanan (RTM): dari keperluan ke bukti

Requirements Traceability Matrix: from requirement to evidence

Ke hadapan: setiap keperluan ada ujian. Ke belakang: setiap ujian dan pepijat ada keperluan. KRISAv2 Bab 2 2.11, Templat T2.8.



BR_ID	FR_ID	Keperluan	Test_Case_IDs	Peringkat	Status	Defect_IDs	Liputan
BR-03	REQ-CHK-01	Kuantiti setiap item 1 hingga 10	TC-CHK-001 hingga 004	Sistem	Gagal	BUG-CHK-004	100%
BR-04	REQ-CHK-03	Penghantaran percuma bagi subjumlah RM200 dan ke atas	TC-CHK-008 hingga 012	Sistem	Lulus		100%
BR-06	REQ-CHK-06	Checkout p95 bawah 2 saat, 50 pengguna serentak	(tiada)		Belum diuji		0% jurang

Test Design

From a single well-formed test case to a planned, harnessed and judged set of tests

01 Anatomy of a test case

02 The testing process

03 The test and debug cycle

04 Worked example: sort, then ShopFast

05 Harness, drivers, stubs and oracle

06 Four testing classifiers

07 Limits of black box and white box

08 Myers guidelines, clustering, checklists

Anatomy of a test case

Test case

A test case has two parts: a description of the input data, and a precise description of the correct output for that input (Myers). Without an expected output written down first, running the program is an experiment, not a test.

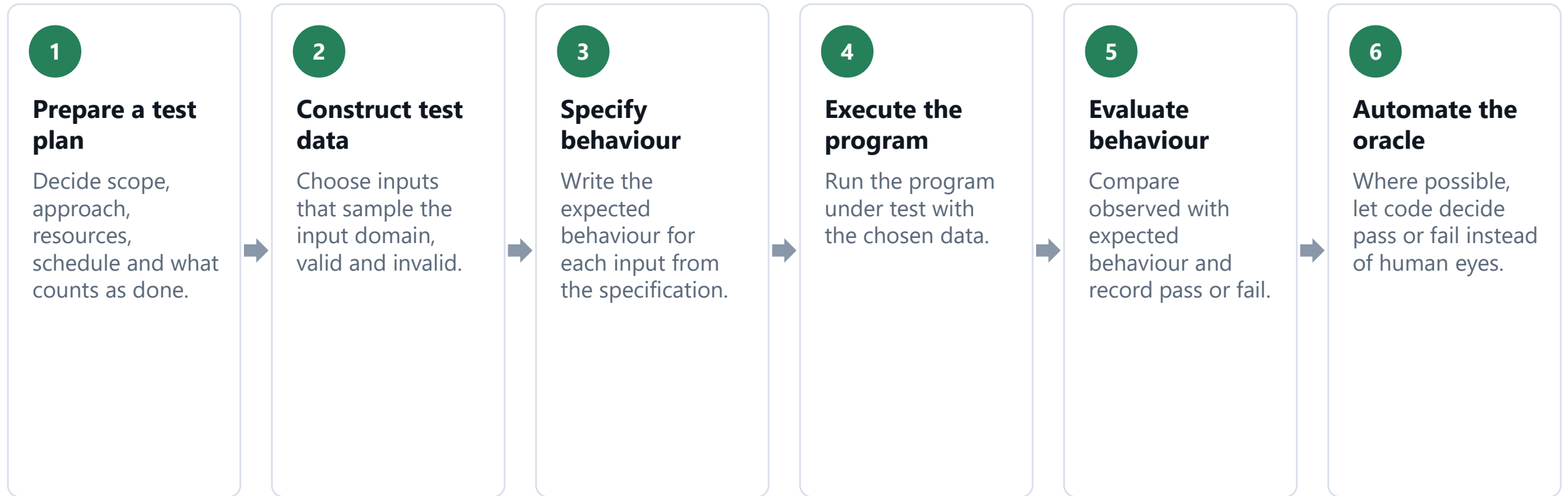
Test data: the values fed to the program under test

Expected result: the outcome the program should produce, taken from the specification

In practice also record preconditions and steps (ISO/IEC/IEEE 29119-3 test case specification)

ShopFast: add P002 qty 2, no code. Expected cart-total RM 78.00 (RM 70.00 plus RM 8.00 shipping)

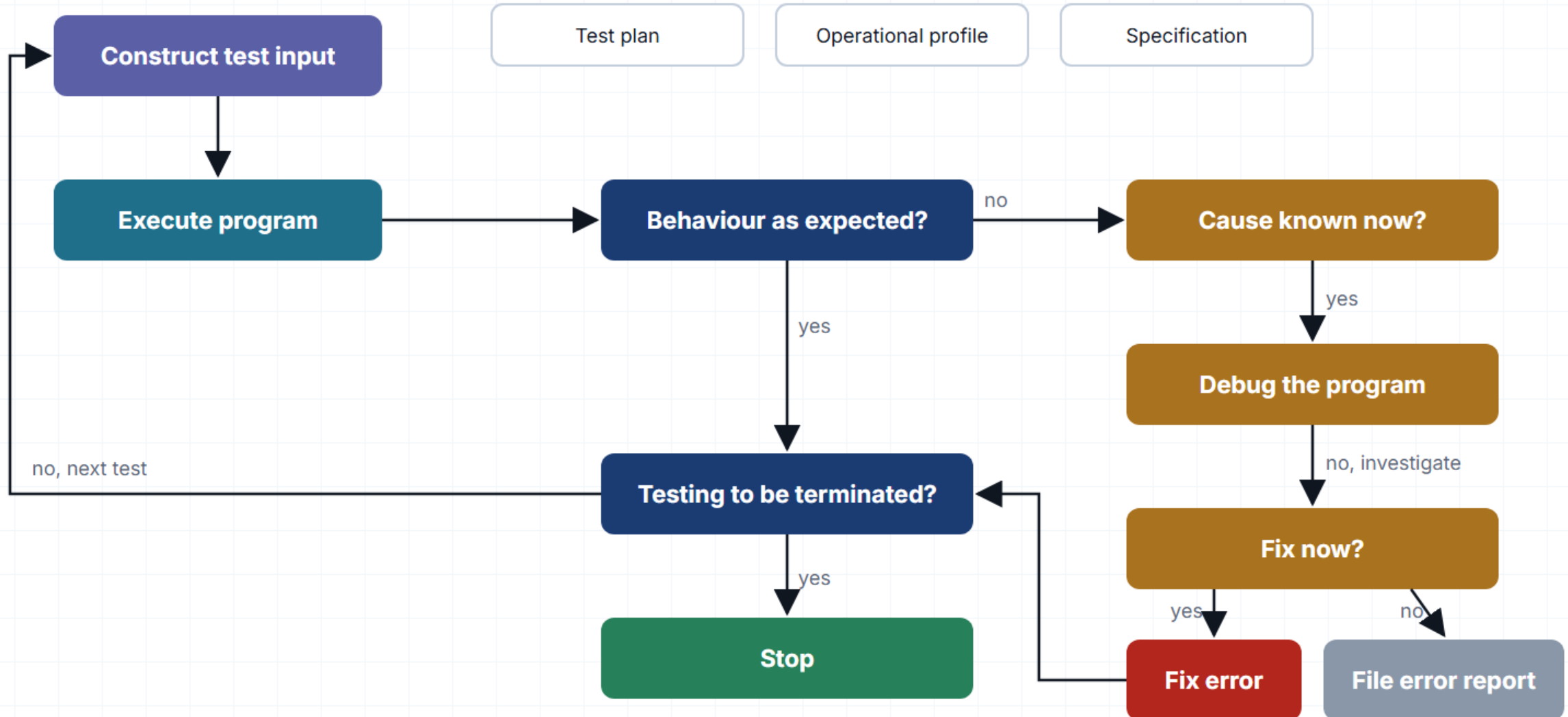
The testing process



The lecture lists execution before specifying behaviour. Write expected results first so the actual output cannot bias you.

The test and debug cycle

Testing finds inputs that cause failures; debugging finds the fault behind a failure



Worked example: the sort program

Sort integers ascending (A) or descending (D). Plan: both orders, empty input, invalid input, report every failure

TC	Test data	Expected output	Purpose
1	A 12 -29 32	-29 12 32	Normal: ascending
2	D 12 -29 32	32 12 -29	Normal: descending
3	A (empty sequence)	No input to be sorted in ascending order	Normal: empty input
4	D (empty sequence)	No input to be sorted in descending order	Normal: empty input
5	R 3 17	Invalid request character	Robustness: bad request character
6	A c 17	Invalid number	Robustness: bad number

The same idea applied to ShopFast

Non-member shopper. Expected results come from the SRS and are written before anything is run

TC	Input	Expected result	Purpose
SF-01	Empty cart, add P002 qty 2	Subtotal RM 70.00, shipping RM 8.00, total RM 78.00	Normal: pricing
SF-02	SF-01 cart, apply RAYA15	Discount RM 10.50, shipping RM 8.00, total RM 67.50	Normal: promo code
SF-03	SF-01 cart, add P002 qty 0	400 VALIDATION_ERROR, cart unchanged	Robustness: invalid quantity
SF-04	SF-01 cart, apply raya15	422 INVALID_CODE_FORMAT	Robustness: lowercase code
SF-05	SF-01 cart, apply ZZZ999	422 INVALID_CODE	Robustness: unknown code
SF-06	Empty cart, add P006 (stock 0) qty 1	400 OUT_OF_STOCK	Robustness: no stock

Test harness, drivers, stubs and the test oracle

The harness runs tests automatically; the oracle decides pass or fail

Test pool

test cases: input plus
expected output

Test harness

- 1 setup (fixture)
- 2 get input from pool
- 3 driver calls the unit
- 4 capture observed output
- 5 check against oracle
- 6 report failure, teardown

Driver: code that calls the unit under test.

Stub: fake of a component the unit calls (for example a payment gateway).

Program under test

ShopFast POST /api/cart/items

observed behaviour

Oracle

Does observed behaviour match expected
behaviour?

Sources of expected results: the specification, a trusted
older version, a calculation, or a human judge.

Four ways to classify testing

Classifier	Question it answers	Techniques	ShopFast example
C1 Source of tests	Where do the test cases come from?	Black box: BVA, partitioning, random, pairwise. White box: structural, path, data flow	Quantity values from the SRS; branch tests of pricing code
C2 Lifecycle phase	When in the life cycle is it done?	Unit, integration, system, regression (maintenance), beta (pre-release)	Unit test of rounding; regression run after every fix
C3 Goal	What property are we after?	Functional, security, robustness, penetration, performance, compatibility	p95 under 2 s with 50 concurrent users (REQ-CHK-06)
C4 Artifact under test	What kind of software is it?	OO, web, real-time, concurrency, database testing	Web application with a REST API

Black box vs white box: both have limits

Black box (from the spec)

- Test data derived solely from the specification, without the code
- To be certain, you would need to try every valid and invalid input
- Programs with memory also need every valid and invalid input sequence
- So exhaustive input testing is impractical for almost all programs
- ShopFast: every mix of products, quantities, codes and sessions is far too many

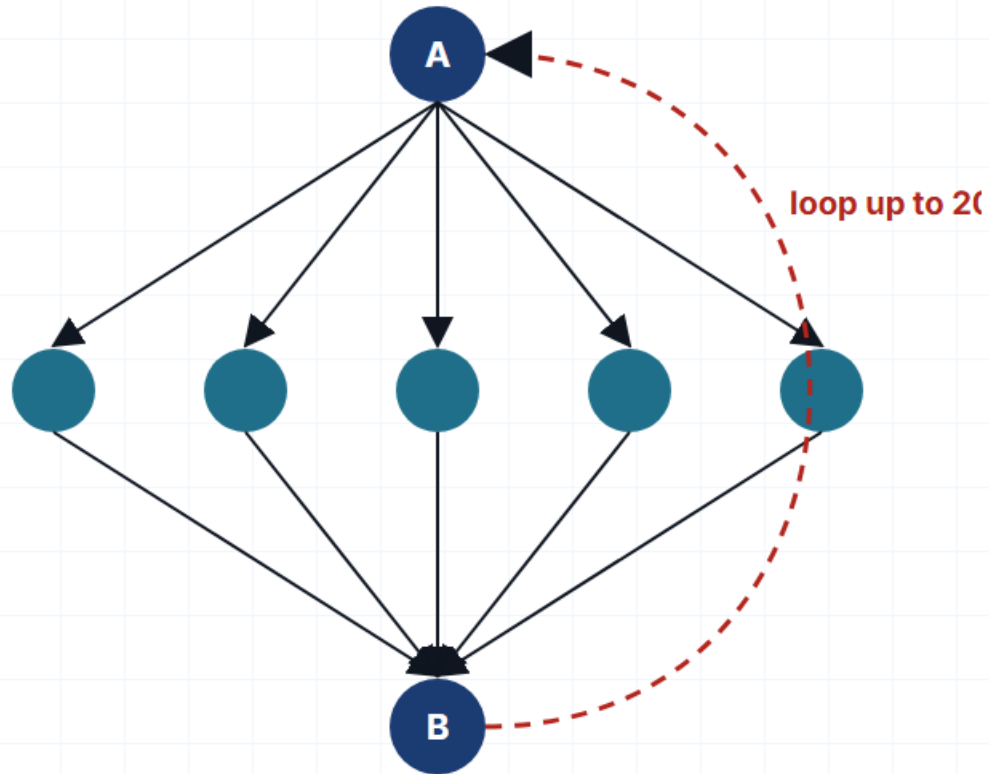
White box (from the code)

- Test data derived by examining the program's logic
- Flaw 1: the number of unique paths is astronomical
- Flaw 2: every path can pass and the program can still be wrong
- Missing paths: absent code cannot be exercised, so it goes unnoticed
- Data sensitivity: some faults appear only for particular values

Neither can be exhaustive. Combine both and select tests with techniques such as partitioning and boundary values.

Why exhaustive path testing is impossible

A small loop with a few branches already has an astronomical number of paths



5 paths through the loop body per iteration

Paths = $5^{20} + 5^{19} + \dots + 5^1$ which is about 10^{14} (100 trillion).

One test designed, run and checked every **5 minutes**: about **1 billion years**.

One test every **millisecond**: still about **3,170 years**.

And testing every path still misses missing paths and data-sensitive errors. So we select tests with techniques and risk.

Every path tested, still wrong

- Two tests execute both branches: full path coverage
- $a=1.0, b=1.0$ gives true and $a=9.0, b=1.0$ gives false: both pass
- $a=0.5, b=9.0$ wrongly returns true because $a - b$ is negative
- The fault is data sensitive: only some values reveal it
- Fix: compare the absolute difference, $\text{fabs}(a - b) < \text{epsilon}$

CONVERGENCE CHECK IN C

```
#include <math.h>
#include <stdbool.h>

/* Spec: converged when a and b differ by less than epsilon */
bool converged(double a, double b, double epsilon)
{
    if ((a - b) < epsilon) { /* fault: should be fabs(a - b) */
        return true;
    }
    return false;
}
```

Myers' basic testing guidelines (1)

- **Two-part test cases:** Every test case states the input data and the precise expected output.
- **Independence:** A programmer should avoid being the only tester of their own program, and an organisation should not only test its own programs.
- **Inspect every result:** Examine each result thoroughly; many failures are visible in the output but missed.
- **Invalid input too:** Write test cases for invalid and unexpected inputs, not only valid and expected ones.
- **Unintended behaviour:** Check that the program does not do what it should not: extra functions and side effects.

Myers' basic testing guidelines (2)

Keep your test cases

Avoid throw-away tests unless the program is throw-away. Tests are an investment reused for regression testing.

Expect to find errors

Never plan a testing effort on the tacit assumption that no errors will be found.

Put strong people here

Testing is creative and challenging, arguably more than design. Do not staff it only with your newest people.

Aim to break it

A good test case is one with a high probability of detecting an as yet undiscovered error.

Defect clustering

Defect clustering

The probability that more errors exist in a section of a program is proportional to the number of errors already found in that section (Myers). Defects are not spread evenly; they tend to gather in a few parts of the system.

Found many bugs in one module? Test it harder instead of moving on

Record defects per module so that clusters become visible

Consider a review or redesign for a module that keeps producing defects

ShopFast: if promo validation yields three bugs, add more promo tests before closing

Checklist: reviewing a test plan

- ☐ Have the major test phases been properly identified and sequenced?
- ☐ Are test resources and tools identified and available?
- ☐ Is traceability to validation criteria and requirements established?
- ☐ Has a test record keeping mechanism been established?
- ☐ Are major functions demonstrated early?
- ☐ Have test drivers and stubs been identified, and has work to develop them been scheduled?
- ☐ Is the test plan consistent with the overall project plan?
- ☐ Has stress testing been specified?
- ☐ Has a test schedule been explicitly defined?

Add stop criteria: the plan should say when testing is finished, for example no open priority 1 defects.

Checklist: reviewing a test procedure

- ☐ Have both white box and black box tests been specified?
- ☐ Are boundary values to be tested?
- ☐ Have all independent logic paths been tested?
- ☐ Are timing and performance to be tested?
- ☐ Have test cases been identified and listed with their expected results?
- ☐ Has acceptable variation from expected results been specified?
- ☐ Is error handling to be tested?
- ☐ ShopFast tailoring: are money results specified to 2 decimal places, exactly as displayed?

Map the partitions and boundaries for ShopFast checkout

GUIDED STEPS

- 1. Open ShopFast-SRS.md and copy the four field requirements into the EP-BVA Analysis tab.
- 2. For each field list the valid partitions first, then every invalid partition. Do not forget empty and wrong-format.
- 3. Mark every boundary. Write the value just below, on, and just above each boundary.
- 4. Remove overlaps: if a boundary value already tests a partition, do not add a second representative.
- 5. Count what remains. Target 8-10 across four fields. Over 14 means you are testing the same value twice.

SHEET TEMPLATE (TAB 1)

Req ID	Field	Partition	Valid?	Value
REQ-CHK-01	Quantity	P1 ≤ 0	N	0
REQ-CHK-01	Quantity	P2 1-10	Y	1 / 10
...	...	...	...	...

TRAINER SPOT-CHECK AT 16:17

- Every field has at least one invalid partition.
- Every range has both a lower and an upper boundary.
- No value appears twice in the sheet.

Write 15-20 test cases **someone else can run**

GUIDED STEPS

- 1. Split the work: each member takes one technique from the case budget.
- 2. Write all case titles first, one line each. Check for duplicates before filling in steps.
- 3. Fill in preconditions, data and steps. One observable expected result per case.
- 4. Set level and priority using the Module 3 Risk Register, not instinct.
- 5. Swap with the next team. They run two of your cases without asking any questions.

PASS

If the reviewer can run it without a single question, the case is done.

TEST CASE SHEET COLUMNS (TAB 2)

TC ID	Req ID	Title	Technique	Priority
TC-CHK-001	REQ-CHK-01	Quantity 0 rejected with a field error	BVA	High
TC-CHK-004	REQ-CHK-01	Quantity 10 accepted into the cart	BVA	High
TC-CHK-008	REQ-CHK-03	Member + RM200 + promo capped at 25%	DT · R1	High
TC-CHK-016	REQ-CHK-05	Cancelling a Shipped order is rejected	ST	Medium

WHAT THE TRAINER CHECKS

Every case has a requirement ID · every case names its technique · at least 5 negative cases · no case depends on the previous one.

Map every case to a requirement, then **hunt the gaps**

GUIDED STEPS

1. Open RTM v0 from Module 2. Add columns: Test Case IDs, Technique, Coverage status.
2. Paste the test case IDs into their requirement row. Every case must land somewhere.
3. Mark empty rows red, those are your coverage gaps. Mark orphan cases amber.
4. Compute coverage and write one sentence: what stays untested, and why that is acceptable.

SHEET FORMULA

=COUNTIF(D2:D20,"<>")/COUNTA(A2:A20)

Format as a percentage, one decimal place.

ANSWERS YOU MUST HAVE AT 16:40

- What percentage of checkout requirements is covered?
- Which high-risk requirement is still uncovered?
- Which test case has no requirement, and should the SRS be updated?
- If only 5 cases could run tonight, which ones and why?

WHY THIS MATTERS LATER

In Module 8 a change request arrives: add discount codes to checkout. This RTM is what tells you in seconds which cases must be re-run, not the whole suite.

Three sample artifacts: **copy the structure, not the content**

01 Test Plan (short form)

M4-ShopFast-TestPlan.md

- ├ 1 Project introduction
- ├ 2 Context: items, levels, entry/exit
- ├ 3 Strategy: techniques, data, environment
- ├ 4 Test risk register
- └ 5 Test deliverables

2 pages. Structure follows the Master Test Plan.

02 Test case sheet

M4-ShopFast-TestCases.xlsx

- ├ tab 1 · EP-BVA analysis
- ├ tab 2 · Test cases (11 columns)
- ├ tab 3 · Decision table
- └ tab 4 · State transitions

18 worked cases are pre-filled as a detail reference.

03 RTM v1

M4-ShopFast-RTM.xlsx

- ├ Req ID · summary · risk
- ├ Test case IDs · technique
- ├ run status · defect IDs
- └ coverage cell · gap list

The same file is reopened in Modules 6, 7 and 8.

KRISA CROSS-REFERENCE FOR THESE ARTIFACTS

Test Plan to [D12](#) Master Test Plan ([Ch.6](#)). Test cases and the RTM support the [D11](#) System Test Report ([Ch.5](#)) and feed the Acceptance Test Report (F5.6).

RULE OF USE

These samples are written for ShopFast. Take the column structure and ID conventions back to your agency; the content must come from your own system SRS.

Six pitfalls that make a test suite **look complete but hollow**

01 · Happy path only

Eighteen cases, all valid input. Real users type odd characters and hit the back button.

Fix: at least 30% negative cases.

02 · Vague expectations

The system should work. Nobody can prove that failed, so the case never finds anything.

Fix: name the exact message, value or screen.

03 · Chained cases

TC-007 only runs if TC-006 passed first. One early failure wipes out half your run.

Fix: every case sets up its own precondition.

04 · Missed boundaries

Testing 5 and 50 but never 1, 10 or 11, exactly where a developer typed < instead of ≤.

Fix: every range contributes 4-6 cases.

05 · Testing the implementation

Check the orders table has status = 2. When the database is refactored the test breaks even though behaviour is correct.

Fix: assert what the user can see.

06 · RTM written on the last day

Filled in backwards to pass the audit. It never reveals a gap because it was built from the cases that already exist.

Fix: update the RTM on the same day.

The artifact is the assessment, there is no exam

Criterion	Adequate	Strong
Case count	15 cases written	18-20, no duplicates
Technique spread	EP + BVA only	All four techniques used and named
Negative cases	3 cases	≥ 5, including a rejected transition
Runnability	Reviewer asks 1-2 questions	Reviewer runs the case with no questions
Traceability	Most cases have a Req ID	100% mapped + coverage computed + gaps stated

EXIT TICKET, WRITE IT ON A STICKY NOTE

- One requirement you could not test, and why.
- One bug you expect your cases to find tomorrow.

WHAT HAPPENS TOMORROW MORNING

Module 5 takes two of your cases and automates them in [Playwright](#) and [Postman](#). Poorly written cases show up immediately.

TAKE HOME TONIGHT

The test case sheet, RTM v1 and the 2-page Test Plan, keep them in the team folder. Modules 6, 7 and 8 start by opening the same files.

From **six sentences** of requirements to evidence of coverage



EVERY ARTIFACT IS REUSED, MODULE 4 DOES NOT END HERE

Module 5 automates the cases Module 6 logs defects against them Module 7 audits the evidence Module 8 scopes regression

KRISA: Test Plan to [D12](#) Master Test Plan ([Ch.6](#)) · cases + RTM support [D11](#) System Test Report ([Ch.5](#)) and the Acceptance Test Report (F5.6)

Techniques: ISO/IEC/IEEE 29119-4 (test techniques) · [ISTQB](#) Foundation Level

References and standards

JDN

Malaysian public sector

- KRISA, Public-Sector Application Engineering, [Ch.6](#) Acceptance Testing ([D12](#) Master Test Plan; processes F5.1, F5.3-F5.6).
- Public-Sector Software Quality Assurance Portal, sqa.jdn.gov.my
- [PPriSA](#), Public-Sector ICT Project Management guideline.

KRISA 2.0 is Beta 2026; deliverable numbering is still being refined. Confirm which version your agency uses.

MODULE FILES

M4-ShopFast-TestPlan.md
M4-ShopFast-TestCases.xlsx
M4-ShopFast-RTM.xlsx
ShopFast-SRS.md (from Module 2)

INTERNATIONAL

Testing standards

- ISO/IEC/IEEE 29119, software testing standards series; Part 3 test documentation, Part 4 test techniques.
- [ISTQB](#) Certified Tester Foundation Level, the official syllabus for EP, BVA, decision tables and state transition.
- ISO/IEC 25010, product quality model; the source for non-functional requirements tested in PAT.

TRAINER

Al-Abrar bin Abdul Wahid

SQA Trainer • 2-Day Hands-On Workshop • aligned to KRISA (JDN)

abrar.abdulwahid@gmail.com • +60 13-283 5415

Abbreviations used in this module (1 of 5)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
AI	Artificial Intelligence	Software that can answer questions and write text or code by learning patterns from large amounts of data.
API	Application Programming Interface	A doorway that lets one program ask another program for data or an action, without using a screen.
BR	Business Rule	A rule set by the business, for example who may view an order. Used in IDs such as BR-07.
BRS	Business Requirements Specification	The document that states what the business needs, written in business language. KRISA deliverable D02.
BVA	Boundary Value Analysis	Testing at the edges of the allowed values, such as 0, 1, 10 and 11, because mistakes often hide there.
CI	Continuous Integration, or Configuration Item	Two meanings. In DevOps, every code change is built and tested automatically. In configuration management, it is any controlled item such as a document or code file.
CI/CD	Continuous Integration and Continuous Delivery	An automated pipeline that builds, tests and prepares every change for release.
CSV	Comma-Separated Values	A simple table file that opens in Excel, with commas between the columns.
DT	Decision Table	A table of condition combinations, with the expected result for each combination.

Abbreviations used in this module (2 of 5)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
E2E	End-to-End	A test that runs a full user journey through the real screens and services.
EP	Equivalence Partitioning	Grouping inputs that behave the same way, then testing one value from each group.
FAT	Factory Acceptance Test	The formal test with the customer before the system is installed at the live site.
FR	Functional Requirement	Something the system must do. Used in IDs such as FR-03.
ICT	Information and Communication Technology	Computers, networks and software used to handle information.
ID	Identifier	A unique label, such as REQ-CHK-01, so that each item can be traced.
IEC	International Electrotechnical Commission	An international standards body that publishes many software standards together with ISO.
IEEE	Institute of Electrical and Electronics Engineers	A professional body that publishes software engineering standards such as IEEE 730 and IEEE 829.
ISO	International Organization for Standardization	Publishes international standards such as ISO 9001 and ISO/IEC 25010.

Abbreviations used in this module (3 of 5)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
ISTQB	International Software Testing Qualifications Board	Offers the best-known certifications for software testers.
JDN	Jabatan Digital Negara	The Malaysian government department that publishes KRISA and PPrISA.
KRISA	Kejuruteraan Sistem Aplikasi Sektor Awam	The Malaysian public sector guide for engineering application systems. KRISAv2 is the 2026 beta edition.
MIT	Massachusetts Institute of Technology	A university in the United States. Source of the Leveson course notes.
OO	Object-Oriented	A way of writing programs as objects that hold both data and behaviour.
PAT	Provisional Acceptance Test	The KRISA acceptance test done before provisional acceptance of the system, including performance checks.
PDF	Portable Document Format	A document file that looks the same on every computer.
PPrISA	Panduan Pengurusan Projek ICT Sektor Awam	The Malaysian public sector guide for managing ICT projects. Codes such as PPrISA05 are its templates.
PS	PowerShell prompt	When a slide shows PS>, type that command in Windows PowerShell.

Abbreviations used in this module (4 of 5)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
QA	Quality Assurance	Preventing defects by setting up good processes and checking that people follow them.
REQ	Requirement	Prefix in requirement IDs such as REQ-CHK-01. CHK stands for checkout.
REST	Representational State Transfer	A common style of web API that uses web addresses and actions such as GET and POST.
RM	Ringgit Malaysia	Malaysian currency.
RTM	Requirements Traceability Matrix	A table linking each requirement to its design, tests and defects, so nothing is missed.
SDS	Spesifikasi Reka Bentuk Sistem	The System Design Specification. KRISA deliverable D04.
SF	ShopFast	Prefix for ShopFast test case IDs such as SF-01.
SIT	System Integration Test	Testing that the parts of a system, and the systems it connects to, work correctly together.

Abbreviations used in this module (5 of 5)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
SKU	Stock Keeping Unit	A unique code for each product in stock, such as SKU-88.
SQA	Software Quality Assurance	All the planned work that gives confidence the software meets its quality requirements.
SRS	Spesifikasi Keperluan Sistem	The System (or Software) Requirements Specification: what the system must do. KRISA deliverable D03.
ST	State Transition testing	Testing how the system moves between states, for example Paid, Shipped and Cancelled.
SUT	System Under Test	The system, or the part of it, that you are testing.
TC	Test Case	Prefix in test case IDs such as TC-CHK-004.
UAT	User Acceptance Test	Users try the system to confirm it meets their needs before they accept it.
UI	User Interface	The screens, buttons and forms that people use.

References and templates for this module

Every line opens the official JDN document at the exact page. Templates download as Word or PDF.

WHERE IT IS IN THE GUIDES

- Bab 5, 5.7 Pengujian Sistem, peringkat ujian
page 33 · KRISAv2 Bab 5 Fasa Pembangunan
- Bab 6, 6.6 Pelan Induk Pengujian F5.1
page 4 · KRISAv2 Bab 6 Fasa Pengujian Penerimaan
- Bab 6, Jadual 6.2 Peringkat UAT, PAT, FAT
page 5 · KRISAv2 Bab 6 Fasa Pengujian Penerimaan
- Bab 6, 6.7 Dokumentasi Persediaan Ujian F5.2
page 9 · KRISAv2 Bab 6 Fasa Pengujian Penerimaan
- Bab 6, Kes ujian aras tinggi dan rendah
page 11 · KRISAv2 Bab 6 Fasa Pengujian Penerimaan
- Bab 6, 6.8 UAT, kriteria masuk dan keluar
page 20 · KRISAv2 Bab 6 Fasa Pengujian Penerimaan

TEMPLATES TO USE

- D12
Pelan Induk Pengujian

WORD

PDF
- D13
Pelan Ujian Penerimaan (UAT/PAT)

WORD

PDF
- PPrISA05
Pelan Pengurusan Kualiti

WORD

PDF

M04 · W04 · 15 MINUTES · Hari 1, 16:40 hingga 16:55

Hands-On SQA-AI Assistant Workshop

AI test design: EP, BVA and coverage gaps

Generate D13-format test cases with named techniques for two checkout requirements, then ask the AI to find RTM coverage gaps.

TOOL

opencode 1.18 in the lab folder · model: Big Pickle via OpenCode Zen (free, no API key) · synthetic ShopFast data only

[Step-by-step guide \(web\)](#)[Workshop slides \(W04\)](#)[Setup guide W00](#)[opencode docs](#)

AI test design: EP, BVA and coverage gaps

STEPS

- 1 Run /testcases for REQ-CHK-01 and REQ-CHK-03
- 2 Approve the CSV and check boundaries 0, 1, 10, 11 and RM200.00
- 3 Ask the Plan agent for RTM coverage gaps
- 4 Merge the good cases into your Lab B sheet

OUTPUT TO SAVE

`outputs/m04-testcases.csv`

TYPE THIS (opencode> in opencode, PS> in PowerShell)

```
opencode> /testcases REQ-CHK-01 dan REQ-CHK-03, maksimum 10 kes ujian  
  
opencode> Bandingkan outputs/m04-testcases.csv dengan outputs/m02-rtm.csv.  
Senaraikan keperluan tanpa kes ujian dan kes ujian tanpa keperluan. Jangan  
tulis fail.
```

HUMAN CHECK BEFORE YOU ACCEPT IT

- Boundary values 10 and RM200.00 are present
- Every case names its technique and REQ-CHK ID
- Expected results come from the SRS, not from what the app does

Open step-by-step guide

Module hub

Pembantu AI dalam SQA: AI mendraf, manusia bertanggungjawab

AI assistant in SQA: AI drafts, humans remain accountable

Aliran kerja setiap bengkel Hands-On SQA-AI Assistant Workshop menggunakan opencode

1

Konteks

@SRS, @templat,
AGENTS.md

2

Prompt

arahan /review-srs

3

Draf AI

opencode + model

4

Semak manusia

banding dengan templat
KRISA/PPrISA

5

Simpan bukti

outputs/ + git commit

Guna AI untuk

Draf pelan dan laporan, cadang kes ujian
EP/BVA, jana skrip ujian, ringkaskan log, cari
jurang RTM.

Jangan

Tampal data sebenar agensi, terima output
tanpa semak, biar AI reka ID keperluan, kongsi
sesi awam.

Bukti audit

Simpan prompt, output dan semakan dalam git
supaya jejak audit lengkap (Modul 7).